

Progetto di Intelligent Robot Navigation

Relazione Tecnica

Gruppo 01

Noemi Biancamano Alex Cuciniello Davide D'Acunto Federica De Maio

Sommario

In questo documento viene presentata la relazione tecnica del progetto di Intelligent Robot Navigation del gruppo 01. L'obiettivo posto è lo sviluppo di un sistema di navigazione intelligente per TurtleBot4, il quale permetta a questo di muoversi autonomamente in un ambiente conosciuto, alla ricerca di un *marker* specifico. Da lì, il robot deve essere in grado di seguirlo, evitando ostacoli e operando anche in presenza di brevi occlusioni. Il tutto deve essere realizzato nel minor tempo possibile all'interno di un percorso predefinito, con l'obiettivo di minimizzare la funzione temporale specificata di seguito:

$$\begin{aligned} T_{\text{tot}} &= \alpha \cdot t_{\text{espl}} + \beta \cdot t_{\text{goal}} + (N_{\text{penalty}} \cdot t_{\text{penalty}}), \\ \alpha &< \beta, \quad N_{\text{penalty}} \geq 0. \end{aligned} \tag{1}$$

Indice

1	Introduzione	2
2	Nav2 Framework	3
2.1	Behavior Tree custom	3
2.2	Path planner	5
2.3	Path controller	6
2.4	Velocity smoother e controllo delle collisioni	7
3	Architettura	9
3.1	Aruco_Reader	9
3.2	Esplorazione	11
3.2.1	Strategie di ricerca	11
3.3	Inseguimento	14
3.3.1	Nodo <code>compute_pose</code>	14
3.3.2	Kalman filter	17
3.3.3	FSM di inseguimento	18
4	Test e miglioramenti futuri	21

1 Introduzione

L'inseguimento di un *marker* da parte di un robot è una problematica comune nell'ambito della robotica mobile, molto presente in letteratura e con un ampio ventaglio di applicazioni pratiche. In questo progetto alcune idee sono state prese da lavori precedenti, tra le quali citiamo:

- **ArUco e OpenCV:** come *marker* si è scelto di adoperare un ArUco, tipicamente utilizzato in applicazioni di *Computer Vision* e supportato da OpenCV, una libreria open-source per la visione artificiale.
- **Nav2:** in supporto alla navigazione del robot, Nav2 permette di automatizzare il processo di pianificazione del percorso, il ricalcolo in caso di ostacoli imprevisti e l'esecuzione del movimento, grazie a un'architettura modulare ed efficiente rispetto a soluzioni completamente personalizzate.

Queste sono state le scelte principali che hanno guidato lo sviluppo del progetto. Verranno quindi descritte in dettaglio le loro integrazioni all'interno del sistema, insieme alle componenti sviluppate ad hoc per il raggiungimento dell'obiettivo.

Il progetto, e quindi anche il documento, suddivide il problema in due aree principali: **Esplorazione** e **Inseguimento**. Nella prima, il robot deve esplorare l'ambiente alla ricerca del *marker*; nella seconda, una volta individuato, deve seguirlo fino al raggiungimento dell'area obiettivo, evitando ostacoli e operando anche in presenza di brevi occlusioni.

All'interno del progetto, Nav2 costituisce il livello software a cui sono stati delegati i compiti di navigazione del robot, dalla generazione dei percorsi fino alla loro esecuzione. Il framework svolge un ruolo principale in entrambe le fasi.

Nella fase di following, la macchina a stati sviluppata mantiene la responsabilità della logica decisionale, stabilendo quando inseguire una posa visibile del marker, quando utilizzare una posa predetta e quando entrare nello stato di perdita del target. I goal generati vengono quindi inviati a Nav2 e gestiti dal *BT Navigator* tramite l'action `NavigateToPose`, specificando un *Behavior Tree* dedicato al following.

Behavior Tree usato in BT_search.xml e BT_follow.xml
I due file differiscono solo nei commenti: la logica eseguita e' la stessa



Il Behavior Tree implementato è organizzato attorno a un nodo principale di recovery, *NavigateRecovery*, configurato con un numero massimo di tentativi pari a sei. Al suo interno,

la navigazione ordinaria è descritta da una *PipelineSequence*, che combina il calcolo del percorso e la sua esecuzione. Il path globale viene calcolato tramite `ComputePathToPose`, usando il planner `GridBased`, ed è inserito all'interno di un `RateController` a 1 Hz. In questo modo il percorso può essere ricalcolato periodicamente durante l'avanzamento del robot, rendendo la navigazione più robusta rispetto a variazioni della costmap o a piccoli cambiamenti del contesto operativo.

La prima logica di recupero interviene quando il calcolo del path fallisce. In questo caso viene eseguita una pulizia della *global costmap*, seguita da un'attesa di 0.5 s. Questa recovery è stata introdotta perché, durante le prove, la costmap poteva rimanere condizionata da ostacoli non più presenti o da informazioni non aggiornate, impedendo al planner di trovare un percorso valido anche quando fisicamente lo spazio era attraversabile. La pulizia della costmap consente quindi di eliminare questi residui e di effettuare un nuovo tentativo di pianificazione su una rappresentazione più coerente dell'ambiente. L'attesa successiva evita di rilanciare immediatamente il planner prima che la mappa abbia avuto il tempo di aggiornarsi.

La seconda logica di recupero è associata all'esecuzione del path. Il comando `FollowPath`, affidato al controller configurato come `FollowPath`, è preceduto da una condizione `IsStuck`: se Nav2 rileva che il robot non sta progredendo correttamente, l'esecuzione viene considerata fallita e viene attivata la recovery locale. Questa consiste nella pulizia della *local costmap*, in una breve attesa e in una rotazione sul posto di circa 3.14 rad. La pulizia locale si è rivelata utile nei casi in cui ostacoli temporanei o letture non più valide continuavano a influenzare il controller, generando traiettorie troppo conservative o impedendo il movimento. Lo *spin*, invece, ha avuto una funzione più fisica e percettiva: ruotando il robot, è stato spesso possibile uscire da situazioni in cui il robot rimaneva incastrato vicino a un ostacolo e, allo stesso tempo, aggiornare la percezione laser dell'ambiente da una nuova orientazione. Dopo queste operazioni, il sistema poteva ricalcolare un path più pulito e tentare nuovamente il raggiungimento del goal.

Se anche le recovery locali non sono sufficienti, il Behavior Tree attiva una recovery più generale. Questa parte utilizza un `ReactiveFallback`: se il goal viene aggiornato, la recovery viene interrotta e la navigazione può ripartire verso il nuovo obiettivo; altrimenti viene eseguito un `RoundRobin` che alterna una pulizia completa delle costmap locale e globale con una rotazione sul posto. Questa logica rappresenta un tentativo più aggressivo di sbloccare la navigazione, pensato per casi in cui il problema non sia limitato al controller locale ma coinvolga anche la rappresentazione globale dell'ambiente o una configurazione sfavorevole del robot.

Una possibile alternativa allo *spin* sarebbe stata l'utilizzo dell'azione di *backup*, cioè una breve retromarcia prima del nuovo tentativo di pianificazione. In linea teorica questa soluzione poteva risultare più efficace e più rapida: arretrando, il robot avrebbe potuto uscire dalla zona occupata o ad alto costo che stava causando il blocco, liberando spazio davanti a sé e permettendo successivamente un ricalcolo del path da una configurazione più favorevole.

Durante le prove, tuttavia, il *backup* si è mostrato meno affidabile di quanto atteso. La manovra di retromarcia era spesso soggetta a fallimento, probabilmente a causa dei controlli di sicurezza che l'hanno resa non realizzabile entro i vincoli impostati. Il comportamento di *backup* veniva bloccato prima di produrre un reale disimpegno.

Per questo motivo, nella versione finale del Behavior Tree è stato preferito lo *spin*. Questa scelta non rappresenta necessariamente la soluzione più efficiente: la rotazione sul posto è più lenta di una semplice retromarcia e può a sua volta fallire se il robot si trova già troppo vicino a un ostacolo. Tuttavia, nelle prove effettuate si è rivelata più stabile del *backup*, perché permetteva spesso di modificare l'orientamento del robot, aggiornare la percezione dell'ambiente e ripulire la costmap da ostacoli non più presenti. Dopo la combinazione di *clear costmap* e *spin*, il planner riusciva in molti casi a calcolare un nuovo percorso valido verso il goal.

2.2 Path planner

Il *planner server* è il componente di Nav2 incaricato di calcolare il percorso globale tra la posa corrente del robot e il goal di navigazione. Nel progetto sono stati configurati due planner, **GridBased** e **Smac2D**, ma nella navigazione online eseguita dai Behavior Tree è stato utilizzato **GridBased**. Questa scelta è esplicitata nei file `BT_search.xml` e `BT_follow.xml`, dove il nodo `ComputePathToPose` richiama il planner con il parametro `planner_id="GridBased"`.

Il planner **GridBased** fornisce i seguenti parametri: `tolerance=0.5`, `use_astar=false`. Il primo parametro consente al planner di accettare un percorso che termini entro una certa tolleranza dal goal, rendendo la pianificazione più robusta in presenza di celle occupate o difficili da raggiungere esattamente. Il parametro `use_astar=false` indica l'utilizzo della modalità classica (Dijkstra).

Parallelamente è stato configurato e testato anche **Smac2D**, basato su `nav2_smac_planner/SmacPlanner2D`. Durante le prove, questo planner tendeva a generare traiettorie più centrali rispetto ai corridoi e quindi più distanti dagli ostacoli laterali. Questo aspetto risultava vantaggioso soprattutto nella fase di ricerca, perché riprendere la navigazione dal centro del corridoio permetteva alla camera di osservare meglio l'ambiente davanti a sé per rilevare eventuali traget posti ai margini dei corridoi.

Tuttavia, lo stesso comportamento si è rivelato anche una criticità. **Smac2D** risultava più restrittivo rispetto alla presenza di ostacoli e ai costi della mappa; in alcuni casi, soprattutto in passaggi stretti o in zone in cui la costmap presentava celle ad alto costo, il planner tendeva a non generare alcun percorso valido. Questo causava blocchi nella navigazione anche in situazioni in cui il robot avrebbe potuto fisicamente attraversare il passaggio. Per questo motivo, nella configurazione finale dei Behavior Tree è stato preferito **GridBased**.

La pianificazione dipende direttamente dalla *global costmap*, configurata nel frame `map`. La costmap globale viene aggiornata a 1.0Hz e pubblicata alla stessa frequenza, con risoluzione pari a 0.05m e `robot_radius=0.19m`. Sono stati attivati i layer `static_layer`, `obstacle_layer` e `inflation_layer`. Lo `static_layer` utilizza la mappa dell'ambiente, mentre l'`obstacle_layer` integra le misure provenienti dal laser scan. `obstacle_max_range=3.2m` e `raytrace_max_range=4.0m`. L'`inflation_layer` è stato impostato con `inflation_radius=0.30m` e `cost_scaling_factor=7.0`, in modo da aumentare il costo delle celle vicine agli ostacoli e mantenere un margine di sicurezza durante la pianificazione.

Anche la *local costmap*, pur essendo principalmente coinvolta nel controllo del moto, influenza il comportamento complessivo del sistema. Essa è configurata nel frame `odom`. La frequenza di aggiornamento è più alta rispetto alla costmap globale, pari a 10.0Hz, con pubblicazione a 5.0Hz. I layer utilizzati sono `static_layer`, `voxel_layer` e `inflation_`

layer. Il `voxel_layer` elabora i dati del laser scan per rappresentare gli ostacoli locali, mentre l'inflation locale è stata configurata con `inflation_radius=0.30m` e `cost_scaling_factor=8.0`.

I valori relativi a raggio del robot, raggio di inflation, soglie di costo e range dei sensori sono stati definiti sperimentalmente. L'obiettivo era trovare un compromesso tra sicurezza e attraversabilità: aumentare troppo l'inflation o rendere il planner eccessivamente conservativo portava il robot a rifiutare passaggi ancora fisicamente percorribili, mentre ridurre eccessivamente i margini avrebbe aumentato il rischio di avvicinamento agli ostacoli. La configurazione finale privilegia quindi la robustezza del raggiungimento del goal, accettando percorsi meno centrali e più rischiosi dal punto di vista delle collisioni gestite però dagli altri moduli dello stack.

2.3 Path controller

Il *controller server* è il componente di Nav2 incaricato di trasformare il percorso globale calcolato dal planner in comandi di velocità per il robot. Nel progetto il controller è richiamato dal nodo `FollowPath` presente nei Behavior Tree, utilizzando il plugin identificato con `controller_id="FollowPath"`. La configurazione finale utilizza `nav2_mppi_controller::MPPIController`, scelto dopo diverse prove sperimentali con controller differenti. L'MPPI ha fornito i risultati migliori perché espone un numero elevato di parametri e di *critics*, permettendo di intervenire in modo più fine sul compromesso tra inseguimento del path, distanza dagli ostacoli, orientamento verso il goal e regolarità del moto.

Il controller è stato configurato con frequenza pari a 10.0Hz e con modello di moto `DiffDrive`, coerente con la cinematica del TurtleBot. I limiti principali di velocità sono stati impostati a `vx_max=0.75`, `vx_min=-0.10`, `vy_max=0.0` e `wz_max=1.2`. Il valore nullo della velocità laterale riflette l'impossibilità del robot di muoversi lateralmente, mentre la piccola velocità negativa consente solo limitate correzioni in retromarcia. L'MPPI valuta un insieme di traiettorie campionate, configurato con `time_steps=40`, `model_dt=0.10` e `batch_size=600`; in questo modo il controller considera un orizzonte temporale sufficiente a reagire agli ostacoli locali senza rendere il calcolo eccessivamente pesante causando exceed del rate e quindi comportamenti anomali.

Una parte centrale della configurazione riguarda i *critics*. Il `CostCritic` penalizza le traiettorie che attraversano celle costose della costmap, con `cost_weight=3.5`, `critical_cost=300.0` e `collision_cost=1000000.0`. Questo critic evita che il robot segua il path globale in modo troppo rigido quando localmente sono presenti ostacoli o celle ad alto costo. Il `PathAlignCritic` e il `PathFollowCritic` regolano invece quanto il robot debba rimanere aderente al percorso calcolato, con pesi rispettivamente pari a 16.0 e 9.0.

Sono stati inoltre attivati il `GoalCritic`, il `GoalAngleCritic` e il `PathAngleCritic`, utilizzati per migliorare l'avvicinamento al goal e l'orientamento finale del robot. Il `PreferForwardCritic`, con `cost_weight=5.0`, è stato mantenuto per favorire traiettorie in avanti ed evitare comportamenti eccessivamente basati sulla retromarcia, risultati poco affidabili durante le prove. La configurazione è completata dal `ConstraintCritic`, che contribuisce a mantenere le traiettorie generate entro i vincoli cinematici del robot.

Per verificare che il robot stesse effettivamente avanzando, è stato utilizzato un `PoseProgressChecker`, con `required_movement_radius=0.3`, `required_movement_angle=0.5`

e `movement_time_allowance=5.0`. Il raggiungimento del goal è invece valutato tramite `SimpleGoalChecker`, con tolleranza lineare e angolare pari a `0.25`. Anche questi valori sono stati scelti sperimentalmente: tolleranze troppo strette portavano a oscillazioni e tentativi di correzione poco utili vicino al goal, mentre tolleranze troppo ampie riducevano la precisione del raggiungimento.

2.4 Velocity smoother e controllo delle collisioni

Il `velocity_smoother` e il `collision_monitor` sono stati inseriti come ultimo livello della pipeline di navigazione, tra i comandi prodotti da Nav2 e il comando effettivamente inviato alla base robotica. Nel launch file personalizzato, sia il `controller_server` sia il `behavior_server` pubblicano i propri comandi su `cmd_vel_nav`. Questo topic viene elaborato dal `velocity_smoother`, passato al `collision_monitor` e infine pubblicato su `cmd_vel`.

`cmd_vel_nav` → `velocity_smoother` → `cmd_vel_smoothed` → `collision_monitor` →
`cmd_vel`

Il `velocity_smoother` ha il compito di rendere più regolari i comandi di velocità, limitando variazioni troppo brusche tra un comando e il successivo. I limiti di velocità sono stati impostati a `max_velocity=[0.75,0.0,1.2]` e `min_velocity=[-0.10,0.0,-1.2]`. Sono stati inoltre impostati i limiti di accelerazione e decelerazione, rispettivamente `max_accel=[2.5,0.0,2.5]` e `max_decel=[-2.5,0.0,-2.5]`.

Il `collision_monitor` è stato aggiunto con l'obiettivo di rendere più continua e veloce l'esecuzione della navigazione. Durante le prove, quando il robot arrivava troppo vicino a un ostacolo o urtava una zona critica, il sistema tendeva a interrompere il normale avanzamento e ad attivare le recovery definite nel Behavior Tree. Questo comportamento era sicuro, ma introduceva una perdita di tempo significativa: prima di riprendere la navigazione era necessario attendere l'esecuzione delle azioni di recupero, come la pulizia delle costmap, l'attesa e lo *spin*. Per ridurre il numero di ingressi in recovery è stato quindi introdotto il `collision_monitor` come livello intermedio di protezione.

Il nodo riceve in ingresso `cmd_vel_smoothed` e pubblica in uscita `cmd_vel`, modificando il comando qualora venga rilevata una condizione di rischio davanti al robot. Nel progetto è stato abilitato solo il poligono `FrontSlow`, definito come una regione rettangolare frontale lunga circa `0.85m` e larga quanto l'ingombro fisico del TurtleBot. Quando almeno `max_points=3` punti del laser scan ricadono in questa regione, viene applicata l'azione `slowdown` con `slowdown_ratio=0.35`. In questo modo il robot rallenta prima di arrivare troppo vicino all'ostacolo, dando al controller il tempo di correggere la traiettoria o di seguire un path più adatto, senza dover necessariamente entrare in una fase di recovery.

È stato configurato anche un secondo poligono, `FrontStop`, pensato per arrestare il robot in una zona frontale più ravvicinata. Tuttavia, nella configurazione finale questo comportamento è stato disabilitato tramite `enabled=False`. La scelta di non usare una vera e propria stop zone è stata intenzionale: un arresto completo avrebbe potuto bloccare frequentemente la navigazione e causare comunque l'attivazione delle recovery. Al contrario, la slow zone permette un intervento più graduale, rallentando il robot vicino agli ostacoli senza interrompere immediatamente l'esecuzione del path. La sorgente sensoriale utilizzata è `scan`,

abilitata come observation source. Anche questi valori sono stati regolati sperimentalmente: l'obiettivo era ottenere una reazione sufficientemente rapida agli ostacoli frontali, evitando blocchi dovuti a effettive collisioni del robot con gli ostacoli.

3 Architettura

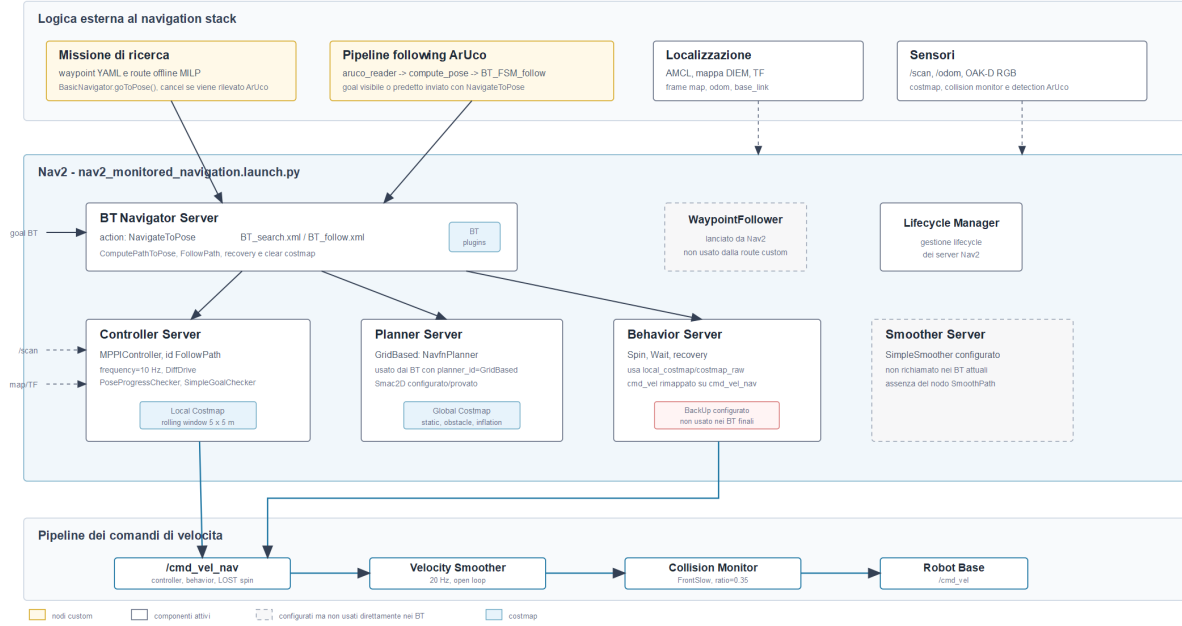


Figura 2: Schema dello stack di navigazione implementato nel progetto.

La Figura 2 riassume l'architettura complessiva del sistema sviluppato, evidenziando l'interazione tra i nodi custom del progetto e lo stack di navigazione Nav2. La logica applicativa è stata organizzata attorno a tre componenti principali: **aruco_reader**, responsabile della rilevazione del marker e della pubblicazione della sua posa, la missione di *search*, incaricata dell'esplorazione dei waypoint fino all'individuazione del target, e la fase di *follow*, che gestisce l'inseguimento del marker una volta rilevato.

Questi tre attori costituiscono il livello decisionale del sistema: producono informazioni, goal e transizioni operative, mentre Nav2 viene utilizzato come livello di navigazione per pianificare, controllare ed eseguire il movimento del robot. Nelle sezioni successive vengono quindi descritti nel dettaglio il ruolo di ciascun componente e il modo in cui essi cooperano per passare dalla ricerca autonoma al following del target.

3.1 Aruco_Reader

Il nodo **aruco_reader** è stato sviluppato utilizzando OpenCV e il modulo ArUco, che permette di rilevare marker all'interno delle immagini acquisite dalla telecamera. Nel progetto il nodo riceve in ingresso il flusso RGB della OAK-D dal topic `/oakd/rgb/preview/image_raw/compressed` e i parametri intrinseci della camera dal topic `/oakd/rgb/preview/camera_info`. Quest'ultimo messaggio è necessario per stimare correttamente la posa tridimensionale del marker a partire dalla sua proiezione nell'immagine.

Il nodo è parametrico rispetto al topic dell'immagine, al topic di **CameraInfo**, al frame della camera, alla dimensione fisica del marker e all'identificativo ArUco da cercare. Nella

configurazione utilizzata, il marker ha lato pari a 0.18m, identificativo 372 e appartiene al dizionario DICT_4X4_1000. Il frame associato alla posa pubblicata è `oakd_rgb_camera_optical_frame`, coerente con il frame ottico della camera.

A ogni frame ricevuto, l'immagine viene convertita tramite `CvBridge` e analizzata con il detector ArUco di OpenCV. Se tra i marker rilevati è presente quello di interesse, il nodo ne stima la posa tramite `estimatePoseSingleMarkers`. La posa stimata viene quindi pubblicata come messaggio `PoseStamped` sul topic `/aruco/pose`. È importante osservare che questa posa è espressa rispetto alla camera, non direttamente nel frame globale della mappa: la trasformazione successiva verso `map` viene gestita dal nodo `compute_pose`.

Per evitare una pubblicazione eccessivamente frequente e ridurre il carico sui nodi successivi, la frequenza massima di uscita è limitata tramite il parametro `output_rate_hz`, impostato a 5.0Hz. Inoltre, il nodo attende di ricevere i parametri della camera prima di elaborare i frame; una volta acquisita la matrice intrinseca e i coefficienti di distorsione, la sottoscrizione a `CameraInfo` viene chiusa, a meno che non sia abilitato il controllo continuo tramite il parametro `always_check_camera_parameters`.

Il nodo gestisce anche il caso di perdita del marker. Dopo un certo numero di frame consecutivi senza rilevamento, pari a `max_marker_lost_frames=5`, viene pubblicato un singolo messaggio `PoseStamped` vuoto, con posizione nulla e orientamento identità. Questo messaggio viene usato dai nodi successivi come segnale di marker non visibile. In particolare, la missione di ricerca ignora le pose vuote e interrompe la navigazione solo quando riceve una posa valida, mentre `compute_pose` interpreta il messaggio vuoto come perdita del target e può attivare la logica di predizione o di *target lost*.

Questa scelta consente di centralizzare la rilevazione del marker in un unico nodo, mantenendo separata la percezione visiva dalla logica decisionale. Il nodo `aruco_reader` si occupa esclusivamente di riconoscere il marker e pubblicarne la posa, mentre la search mission utilizza tale informazione per fermare l'esplorazione e la pipeline di following la usa, tramite `compute_pose`, per generare i goal di inseguimento.

Comunicazione del nodo aruco_reader

Flusso dei topic usati per rilevare il marker e alimentare search e following

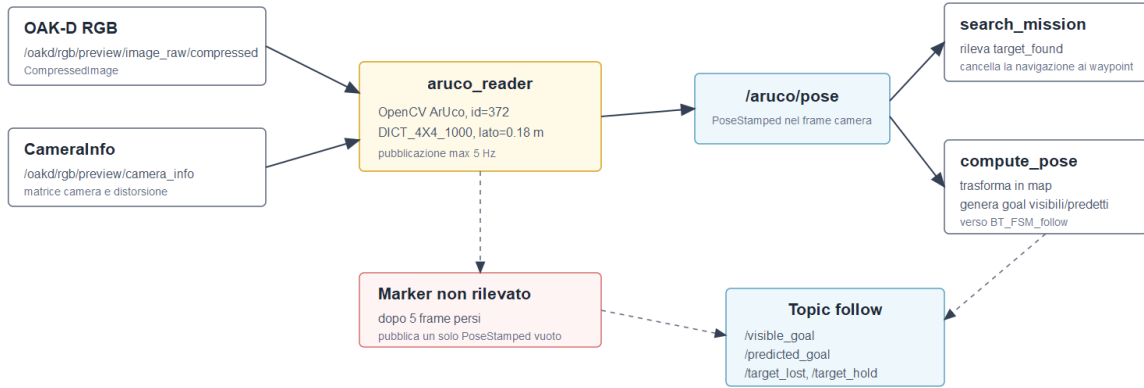


Figura 3: Flusso dei topic associati al nodo **aruco_reader**.

3.2 Esplorazione

La fase di esplorazione è stata progettata con l'obiettivo di esplorare l'intera mappa nel minor tempo possibile, riducendo gli spostamenti ridondanti e massimizzando la probabilità di rilevare il marker durante il percorso. Poiché il robot dispone di una camera frontale con campo visivo limitato, la strategia non consiste semplicemente nel raggiungere una sequenza di punti della mappa, ma nel pianificare una traiettoria che permetta di coprire visivamente le aree di interesse.

Per questo motivo, la mappa viene descritta tramite un insieme di waypoint selezionati in funzione della geometria dell'ambiente, della disposizione dei corridoi e delle zone che devono essere osservate. Alcuni waypoint hanno la sola funzione di transito, mentre altri richiedono una rotazione controllata del robot per estendere la copertura del campo visivo della camera. La ricerca combina quindi navigazione globale, ordinamento dei waypoint e scansioni locali, con lo scopo di coprire l'ambiente in modo efficiente senza visitare inutilmente le stesse aree.

3.2.1 Strategie di ricerca

La strategia di esplorazione è stata costruita separando la descrizione dell'ambiente dalla logica di esecuzione online. Il robot non decide liberamente dove andare durante la missione, ma segue una route costruita a partire da un insieme di waypoint scelti sulla mappa. La scelta progettuale è stata quindi quella di spostare il più possibile il carico computazionale nella fase offline, lasciando online solo le decisioni che dipendono dalla posa iniziale reale del robot e dagli eventuali ostacoli rilevati durante la navigazione.

Mappa e configurazioni operative. Per la fase di ricerca sono state predisposte due configurazioni della mappa. La prima è una configurazione estesa, che contiene l'intero

insieme dei waypoint individuati nell'ambiente. Essa è pensata per coprire in modo completo tutte le aree esplorabili, includendo anche punti più periferici o meno probabili. La seconda è una configurazione ridotta, ottenuta selezionando solo i waypoint ritenuti più significativi per lo scenario di test. Questa versione permette di ridurre il tempo totale di missione e di concentrare l'esplorazione sulle zone più rilevanti.

Entrambe le configurazioni mantengono la stessa struttura logica: i waypoint sono espressi nel frame `map`, sono identificati da un nome stabile e possono essere di tipo `transit` oppure `scan`. I waypoint di tipo `transit` servono solamente a costruire una traiettoria percorribile e non richiedono rotazioni locali. I waypoint di tipo `scan`, invece, rappresentano punti in cui il robot deve orientare la camera verso zone non direttamente visibili lungo il solo percorso di navigazione.

Waypoints e copertura visiva. I waypoint non sono stati scelti come una semplice griglia di punti, ma in funzione della geometria reale dell'ambiente. In particolare, sono stati collocati in corrispondenza di corridoi, incroci e zone da cui la camera del TurtleBot può coprire porzioni significative della mappa. L'obiettivo non è quindi soltanto attraversare l'ambiente, ma massimizzare la copertura del campo visivo durante il movimento.

Questa distinzione è importante perché la camera è frontale e ha un campo visivo limitato. Un waypoint può essere utile anche se non coincide con una possibile posizione del target, purché permetta al robot di osservare un settore nascosto o una diramazione. Per questo motivo alcuni punti sono puramente di transito, mentre altri sono associati a scansioni angolari specifiche.

Calcolo offline dei costi. Per la configurazione estesa è stato utilizzato un approccio offline basato sul planner di Nav2. Invece di stimare la distanza tra due waypoint tramite la distanza euclidea, lo script di calcolo richiede al planner un percorso tra ogni coppia di waypoint e ne misura la lunghezza effettiva. Il costo di un arco rappresenta quindi la distanza percorribile secondo la mappa e la costmap, tenendo conto della presenza di muri, corridoi e passaggi obbligati.

Questo produce una matrice di costi in cui ogni elemento rappresenta il costo pianificato per andare da un waypoint a un altro. Tale matrice è molto più informativa della distanza geometrica diretta, perché due punti vicini in linea d'aria possono richiedere un percorso lungo se separati da pareti o ostacoli strutturali.

Ottimizzazione tramite MILP. Sulla configurazione estesa, le route sono state calcolate risolvendo un problema MILP. Il problema è formulato come un cammino aperto sui waypoint: fissato un possibile waypoint iniziale, il solver deve trovare l'ordine di visita che attraversa tutti i waypoint una sola volta minimizzando la somma dei costi pianificati tra waypoint consecutivi.

Il modello usa variabili binarie per indicare se un arco tra due waypoint viene selezionato nella route e variabili d'ordine per impedire la formazione di cicli interni. La funzione obiettivo minimizza la distanza totale pianificata. Il problema viene risolto una volta per ciascun waypoint considerato come possibile primo nodo; il risultato è un file di route che contiene una route ottimizzata per ogni possibile punto di ingresso nella missione.

Route ridotte costruite manualmente. Per la configurazione ridotta, invece, la route è stata definita manualmente. In questo caso l'obiettivo non era soltanto minimizzare la distanza totale, ma imporre un ordine di esplorazione più coerente con la probabilità di trovare il marker nelle zone centrali e più rilevanti della mappa. Le route manuali permettono quindi di visitare prima le aree considerate più importanti, rimandando eventuali zone secondarie.

Anche in questo caso viene mantenuta la stessa struttura del file di route: per ogni possibile waypoint iniziale esiste una sequenza di visita. Questo consente al codice online di usare la stessa logica indipendentemente dal fatto che la route sia stata prodotta dal MILP o costruita manualmente.

Scelta del primo waypoint. Durante l'esecuzione online, il robot non ricalcola l'intera route. Dopo aver ricevuto la posa iniziale da RViz, calcola soltanto il costo dal punto di partenza reale verso ciascun possibile primo waypoint. In questo modo il robot sceglie quale route offline utilizzare in base alla posizione iniziale effettiva.

La politica di scelta cambia in base alla configurazione utilizzata. Per la mappa estesa, il primo waypoint viene scelto considerando sia il costo per raggiungerlo dalla posa iniziale sia il costo totale della route offline associata a quel primo nodo. In questo modo viene minimizzato il costo complessivo stimato della missione. Per la mappa ridotta, invece, viene considerato solo il costo dal punto iniziale al primo waypoint, perché l'ordine successivo è stato già progettato manualmente per rispettare la strategia di esplorazione desiderata.

Scan angles. I waypoint di tipo `scan` non eseguono tutti una rotazione completa a 360° . Gli angoli di scansione sono stati assegnati in modo mirato, in base alle zone cieche che la camera non riesce a coprire durante il solo avanzamento lungo la route. In alcuni casi è sufficiente osservare un settore laterale, mentre in altri è utile eseguire una rotazione completa.

Gli `scan_angles` sono definiti come angoli relativi allo yaw con cui il robot arriva al waypoint. Il segno dell'angolo stabilisce il verso della rotazione: valori positivi indicano rotazioni antiorarie, valori negativi rotazioni orarie. La rotazione viene eseguita tramite l'action `Spin` del behavior server di Nav2, così il robot può continuare ad ascoltare il topic dell'ArUco anche durante la scansione.

Dipendenza dalla direzione di provenienza. Per alcuni waypoint, il settore da osservare dipende dal lato da cui il robot arriva. Lo stesso punto della mappa può infatti essere raggiunto da direzioni diverse, e un angolo che copre correttamente una zona nascosta in un caso può risultare speculare nell'altro. Per gestire questa situazione, nel file YAML è stato introdotto il campo `invert_scan_angles_from`.

Se il waypoint precedente appartiene alla lista specificata in questo campo, gli angoli di scansione vengono invertiti cambiandone il segno. In questo modo la stessa definizione del waypoint può adattarsi al verso di attraversamento della route, senza duplicare nodi o introdurre logiche specifiche nel codice per ogni singolo caso.

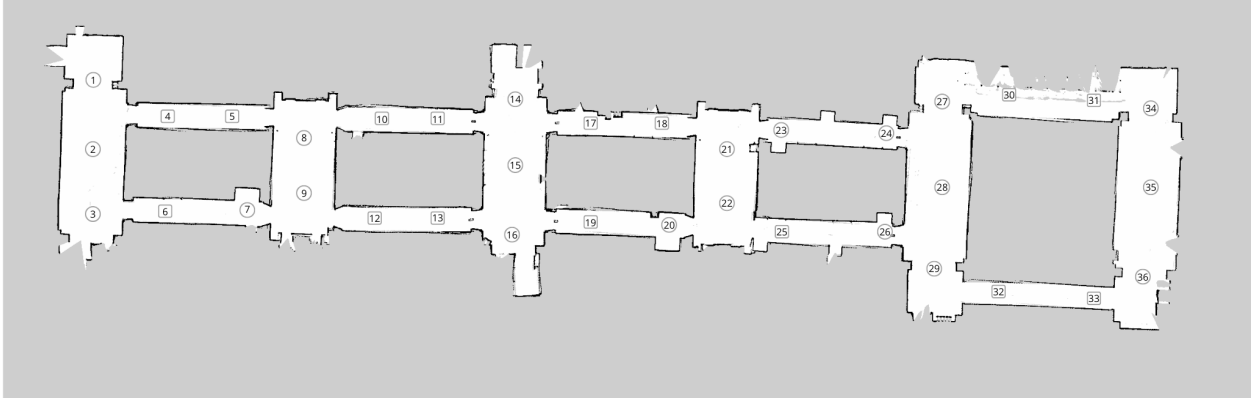


Figura 4: Posizionamento waypoint

- waypoint di *transit*, usato come punto per la rilevazione dell'aruco durante la navigazione;
- waypoint di *scan*, usato per eseguire la ricerca del marker ArUco.

3.3 Inseguimento

Il sistema di inseguimento è organizzato come una pipeline di nodi ROS 2. Il nodo `aruco_reader` acquisisce le immagini della camera, rileva il marker ArUco desiderato e pubblica la sua posa relativa al frame della camera. Tale informazione viene elaborata da `compute_pose`, che trasforma la posa del marker nel frame globale `map`, calcola la posa-obiettivo del robot rispetto al target e distingue tra goal derivati da una misura reale, goal predetti e condizioni di perdita o mantenimento del target.

Le funzioni di supporto raccolte in `literals` forniscono costanti, conversioni geometriche, controlli sulle pose e il filtro di Kalman usato per stimare temporaneamente la posizione del marker in caso di perdita. Infine, la FSM di follow riceve i segnali prodotti da `compute_pose` e decide se inviare un nuovo goal al `bt_navigator`, ignorare un goal ridondante, seguire una predizione oppure entrare nella fase di ricerca locale del marker.

3.3.1 Nodo `compute_pose`

Il nodo `compute_pose` rappresenta il livello intermedio tra la rilevazione del marker e la logica decisionale della FSM. Esso riceve in ingresso la posa dell'ArUco pubblicata su `/aruco/pose` e produce i segnali utilizzati dal nodo di inseguimento per decidere il comportamento del robot.

All'avvio, il nodo inizializza un `KalmanTracker`, crea un buffer TF e rimane in attesa che sia disponibile la trasformazione tra il frame della camera e il frame globale `map`. La camera utilizzata è identificata dal frame `oakd_rgb_camera_optical_frame`. Finché la trasformazione non è disponibile, i messaggi ricevuti vengono ignorati, poiché non sarebbe possibile calcolare una posa-obiettivo coerente nella mappa.

Il nodo pubblica quattro segnali principali:

- `/visible_goal`: goal del robot calcolato a partire da una misura reale dell'ArUco visibile;

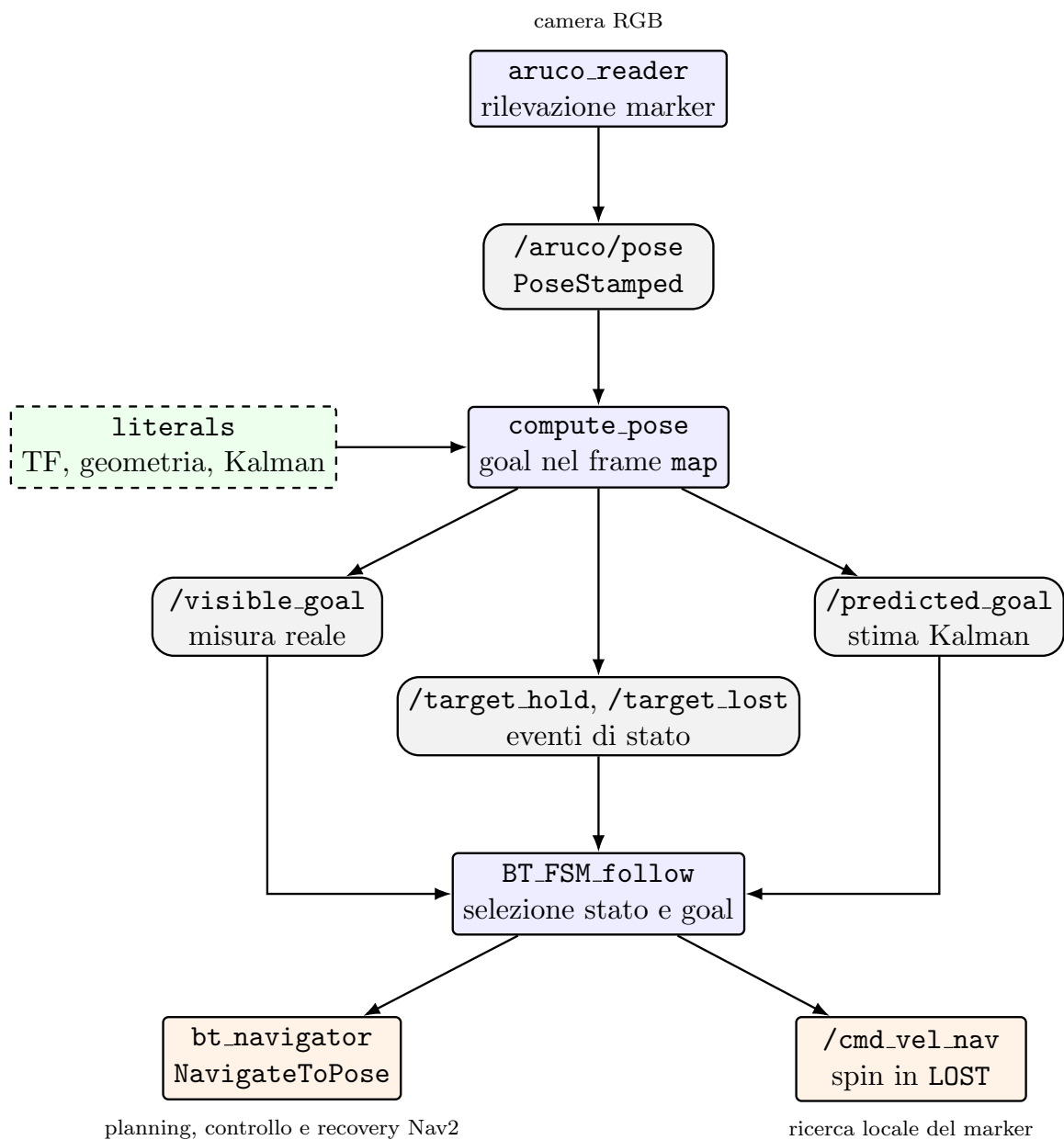


Figura 5: Schema compatto della comunicazione tra i nodi dell'area di inseguimento.

- `/predicted_goal`: goal del robot calcolato a partire dalla posizione predetta del marker tramite filtro di Kalman;
- `/target_lost`: segnale di perdita del marker quando non è disponibile una predizione valida;
- `/target_hold`: segnale usato quando il marker è visibile, ma il robot si trova già in una posizione sufficientemente vicina e ben orientata.

Quando arriva una posa valida dell'ArUco, il nodo la trasforma nel frame `map`. Se la trasformazione TF non è disponibile esattamente al timestamp del messaggio, viene usata la trasformazione più recente, così da evitare di scartare rilevazioni utili a causa di piccoli disallineamenti temporali tra camera e TF. Dopo la trasformazione, la quota verticale viene annullata ponendo $z = 0$, poiché la navigazione avviene sul piano della mappa.

A partire dalla posa del marker in mappa, il nodo calcola la posa-obiettivo del robot. La direzione di approccio viene scelta come il versore che va dal marker verso la posizione corrente del robot. In questo modo il goal viene posto dalla stessa parte da cui il robot sta osservando il marker, evitando di dipendere direttamente dalla normale 3D stimata dell'ArUco, che può risultare rumorosa.

Il goal viene quindi posto a distanza `TARGET_DISTANCE` dal marker lungo questa direzione:

$$d_{\text{target}} = 0.60 \text{ m}$$

L'orientamento finale viene invece calcolato in modo che il robot guardi verso il marker:

$$\theta_{\text{goal}} = \text{atan2}(y_{\text{marker}} - y_{\text{goal}}, x_{\text{marker}} - x_{\text{goal}})$$

Lo yaw così ottenuto viene convertito in quaternion e inserito nella posa pubblicata su `/visible_goal`.

Prima di pubblicare il goal, il nodo verifica se il robot è già abbastanza vicino alla posa-obiettivo. Il controllo usa una tolleranza lineare pari a `0.10 m` e una tolleranza angolare pari a `15 gradi`. Se entrambe le condizioni sono soddisfatte, non viene pubblicato un nuovo goal: il nodo invia invece `/target_hold`. Lo stesso accade se il robot è già più vicino al marker rispetto alla posa-obiettivo calcolata e quindi, per raggiungere il goal, dovrebbe arretrare. In questo caso la retromarcia viene evitata e il target viene considerato già raggiunto.

Quando il marker è visibile, il filtro di Kalman viene aggiornato usando la posa reale dell'ArUco nel frame `map`. Il filtro non viene usato per sostituire la misura reale, ma solo come memoria interna per stimare la posizione del marker in caso di perdita successiva.

Se `aruco_reader` pubblica un messaggio vuoto, il nodo interpreta l'evento come perdita del marker. In questo caso `compute_pose` prova a propagare una sola volta lo stato del filtro di Kalman, ottenendo una posa predetta del marker. Se la predizione è disponibile, viene calcolato un nuovo goal del robot rispetto alla posa predetta e pubblicato su `/predicted_goal`. La predizione viene pubblicata una sola volta per ogni evento di perdita, evitando di generare continuamente nuovi goal stimati mentre il marker resta fuori dal campo visivo.

Se invece il filtro di Kalman non dispone ancora di una misura valida, ad esempio perché il marker non era mai stato osservato correttamente, la predizione non può essere calcolata. In questo caso il nodo pubblica `/target_lost`, lasciando alla FSM il compito di entrare nella fase di ricerca locale del marker.

3.3.2 Kalman filter

Durante l'inseguimento possono verificarsi brevi occlusioni del marker, dovute ad esempio al passaggio di una persona o alla perdita temporanea del target dal campo visivo della camera. Per evitare che il robot entri immediatamente nella fase di ricerca locale a ogni perdita momentanea, è stato introdotto un filtro di Kalman, implementato nella classe `KalmanTracker`.

Il filtro non sostituisce la misura reale quando l'ArUco è visibile. In condizioni normali, infatti, il goal del robot viene calcolato a partire dalla posa reale del marker trasformata nel frame `map`. Il filtro viene aggiornato in background con tali misure e viene utilizzato solo quando il marker viene perso, per produrre una singola stima della sua posizione.

Il modello adottato è un modello lineare a velocità costante. Lo stato interno del filtro è composto da sei variabili:

$$\mathbf{x} = [x \ v_x \ y \ v_y \ \theta \ \omega]^T$$

dove x e y rappresentano la posizione del marker nel frame `map`, v_x e v_y le velocità lineari stimate sul piano, θ lo yaw del marker e ω la velocità angolare stimata. La misura disponibile dalla camera, invece, contiene solo posizione e yaw:

$$\mathbf{z} = [x \ y \ \theta]^T$$

Per questo motivo la matrice di osservazione H seleziona dallo stato solo le componenti misurabili, mentre le velocità vengono stimate implicitamente dal filtro a partire dall'evoluzione temporale delle misure.

A ogni nuova posa valida dell'ArUco, il metodo `update()` esegue due operazioni. Prima propaga lo stato usando il tempo reale trascorso dall'ultima misura, poi corregge la stima con la nuova misura disponibile. Il passo temporale dt non è fisso, ma viene calcolato dai timestamp dei messaggi ROS, così il filtro rimane coerente anche se la frequenza effettiva delle rilevazioni non è perfettamente costante.

La matrice di transizione F viene ricostruita a ogni aggiornamento in base a dt . Per ciascuna componente viene applicato il modello a velocità costante:

$$x_{k+1} = x_k + v_x dt, \quad y_{k+1} = y_k + v_y dt, \quad \theta_{k+1} = \theta_k + \omega dt$$

Il rumore di processo Q viene calcolato assumendo accelerazione lineare e accelerazione angolare come rumore bianco. Nel codice sono utilizzati $\sigma_{acc} = 1.5 \text{ m/s}^2$ e $\sigma_\alpha = 1.5 \text{ rad/s}^2$, così da consentire al filtro di modellare variazioni moderate del moto del marker. Il rumore di misura R , invece, rappresenta l'incertezza della camera: la deviazione standard sulla posizione è pari a 0.05 m, mentre quella sull'orientamento è pari a 35°.

Un aspetto importante riguarda la gestione dello yaw. Poiché gli angoli sono periodici, una variazione da $+\pi$ a $-\pi$ potrebbe apparire numericamente come un salto improvviso. Per evitarlo, prima di aggiornare il filtro viene normalizzata la differenza angolare rispetto allo yaw stimato in precedenza. In questo modo il filtro continua a seguire la stessa direzione geometrica senza introdurre discontinuità artificiali.

Quando il marker non è più visibile, `compute_pose` richiama il metodo `predict_only()`. In questa fase il filtro propaga lo stato usando le velocità stimate, ma non può correggere

la predizione con una nuova misura reale. La posa predetta viene quindi usata una sola volta per calcolare un goal del robot pubblicato su `/predicted_goal`. Se il filtro non ha ancora ricevuto alcuna misura valida, la predizione non è possibile e viene generato il segnale `/target_lost`.

Questa scelta limita l'uso del Kalman alle situazioni in cui è realmente necessario. Quando il marker è visibile, il robot segue la misura reale; quando il marker viene perso, il filtro fornisce una stima temporanea che permette di tentare il recupero del target prima di passare alla rotazione locale di ricerca.

3.3.3 FSM di inseguimento

La logica di inseguimento dell'ArUco è gestita dal nodo `BT_FSM_follow`, che implementa una macchina a stati finiti composta da quattro stati principali: `IDLE`, `FOLLOWING_VISIBLE`, `FOLLOWING_PREDICTED` e `LOST`. La suddivisione in stati permette di gestire l'aggiornamento dei goal da raggiungere e il comportamento del robot in caso di perdita dell'ArUco, mentre la navigazione vera e propria viene affidata al `bt_navigator` tramite l'action `NavigateToPose`.

In questo modo il nodo di follow si occupa della logica decisionale, mentre la gestione del planning, del controllo e delle recovery viene demandata al Behavior Tree di Nav2. Questa scelta consente di sfruttare nativamente il *replanning* dinamico del `bt_navigator`: durante l'esecuzione del goal, il Behavior Tree viene rivalutato periodicamente e può richiedere nuovi percorsi verso la posa-obiettivo corrente. Il robot può quindi adattare la traiettoria alla presenza di ostacoli imprevisti senza che la FSM debba interrompere manualmente l'inseguimento o ricostruire esplicitamente la catena di pianificazione e controllo.

Il nodo riceve quattro segnali principali prodotti da `compute_pose`:

- `/visible_goal`: contiene una posa-obiettivo calcolata a partire da una misura reale dell'ArUco visibile;
- `/predicted_goal`: contiene una posa-obiettivo stimata tramite filtro di Kalman quando il marker viene perso;
- `/target_lost`: segnala la perdita del marker in assenza di una predizione valida;
- `/target_hold`: indica che il marker è visibile, ma il robot si trova già sufficientemente vicino e ben orientato.

Il comportamento della FSM può essere descritto a partire dai singoli stati:

`IDLE`

Il robot non ha un goal di navigazione attivo e rimane in attesa di nuovi comandi. Se riceve un `/visible_goal`, confronta la posa ricevuta con l'ultima posa raggiunta, usando una soglia sulla distanza lineare e sull'errore angolare.

Se il goal è sostanzialmente uguale al precedente, viene scartato per evitare il ciclo indesiderato `IDLE -> FOLLOW -> IDLE`. Se invece il goal è nuovo, viene inviato a `bt_navigator` e lo stato passa a `FOLLOWING_VISIBLE`.

Se arriva un `/predicted_goal`, esso può essere usato come nuovo goal predetto. Un messaggio `/target_hold`, invece, viene ignorato se non esiste alcuno stato di navigazione da interrompere.

FOLLOWING_VISIBLE

Il robot sta inseguendo una posa calcolata da una misura reale dell'ArUco. Se arrivano nuovi `/visible_goal`, il nodo li confronta con il goal attualmente attivo: se la differenza è piccola, il nuovo goal viene scartato; se la differenza è significativa, viene inviato un nuovo `NavigateToPose` al `bt_navigator`.

Le risposte relative a goal precedenti vengono ignorate mediante un contatore interno di sequenza, così da evitare che risultati asincroni vecchi modifichino lo stato corrente. Se durante l'inseguimento visibile arriva un `/predicted_goal`, questo non interrompe immediatamente il goal visibile corrente, ma viene salvato come *pending predicted goal*.

FOLLOWING_PREDICTED

Il robot sta raggiungendo una posa stimata dopo la perdita dell'ArUco. In questa fase un nuovo `/visible_goal` ha priorità sulla predizione: il marker è stato riacquisito, quindi la predizione pendente viene eliminata e il robot può tornare a inseguire la posa reale.

Un messaggio `/target_lost` viene invece ignorato, perché il robot sta già eseguendo una strategia di recupero basata sulla predizione. Se il goal predetto viene raggiunto senza che l'ArUco sia stato riacquisito, il nodo entra nello stato `LOST`; se invece il target è tornato visibile, la navigazione può concludersi tornando in `IDLE`.

LOST

Lo stato rappresenta la perdita effettiva del target. In questa condizione non viene inviato alcun nuovo goal al `bt_navigator`; il robot esegue invece una rotazione sul posto pubblicando periodicamente su `/cmd_vel_nav` una velocità angolare positiva pari a 0.40 rad/s . La pubblicazione avviene tramite un timer a 5 Hz.

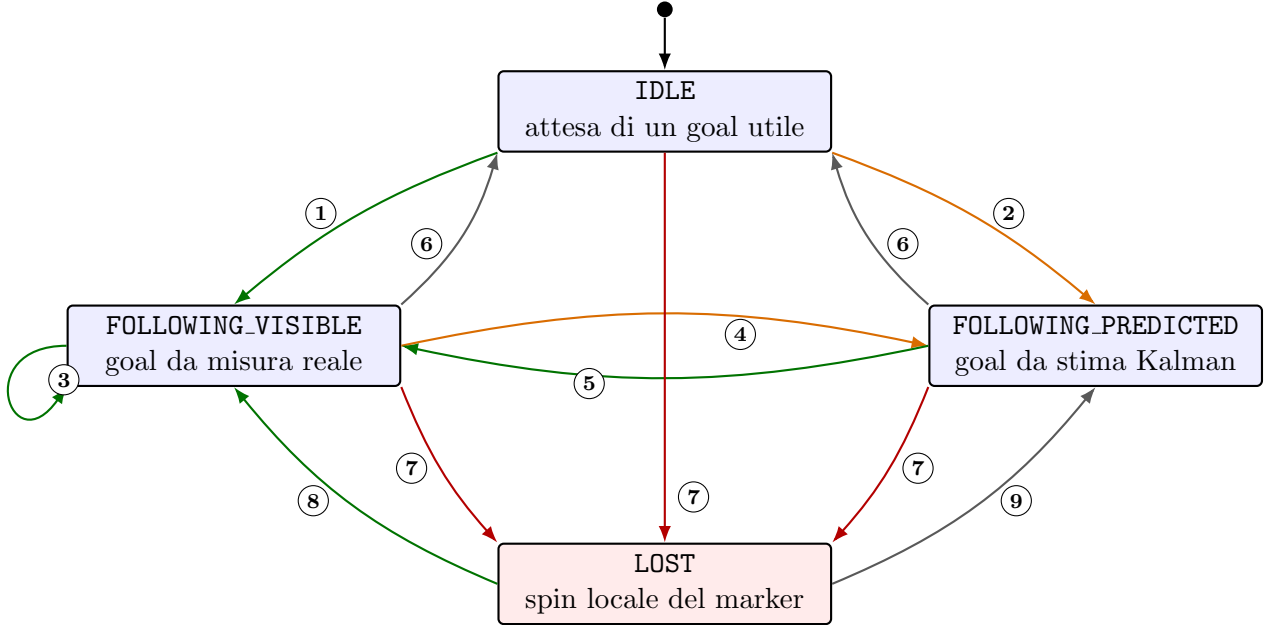
I goal predetti ricevuti mentre il nodo è in `LOST` vengono ignorati, per evitare di seguire stime ormai non più affidabili. Se il marker viene riacquisito, quindi arriva un nuovo `/visible_goal`, lo spin viene fermato, viene pulita la local costmap e solo dopo viene inviato il nuovo goal reale al `bt_navigator`.

Se invece arriva un `/target_hold`, il robot interpreta la situazione come target visibile e già raggiunto, interrompe lo spin e ritorna in `IDLE`.

In caso di successo di una navigazione, il comportamento dipende dalla natura del goal raggiunto e dallo stato di visibilità del target. Se il goal era predetto e l'ArUco non è stato riacquisito, il nodo entra in `LOST`. Se il target non è visibile ma esiste una predizione pendente, questa viene processata prima di dichiarare la perdita definitiva. Se invece il target è visibile, il goal raggiunto viene memorizzato come ultimo goal valido e il nodo torna in `IDLE`.

In caso di fallimento di `NavigateToPose`, il nodo richiede la pulizia della local costmap; successivamente usa un eventuale goal predetto pendente, oppure entra in `LOST` se il marker non è visibile.

Questa FSM separa quindi due responsabilità: da un lato decide quale goal seguire, quando ignorare goal ridondanti e quando dichiarare il target perso; dall'altro lascia al Behavior Tree di Nav2 la gestione della navigazione verso il goal, dell'aggiramento degli ostacoli e delle recovery locali.



- 1 Arriva un `/visible_goal` nuovo: il goal reale viene inviato al `bt_navigator`.
- 2 Arriva un `/predicted_goal`: viene seguito un goal stimato dal filtro di Kalman.
- 3 Durante l'inseguimento reale, un nuovo goal reale significativo sostituisce quello corrente.
- 4 Goal visibile terminato e marker non visibile: viene processato il pending predicted goal.
- 5 La riacquisizione del marker annulla la predizione e riporta al goal reale.
- 6 Il target è già raggiunto o sufficientemente vicino: il nodo torna in IDLE.
- 7 Il marker è perso senza predizione utile, oppure il goal predetto termina senza riacquisizione.
- 8 In LOST, una nuova misura reale ferma lo spin, pulisce la local costmap e riparte.
- 9 In LOST, un `/target_hold` indica target visibile e già raggiunto.

Figura 6: Macchina a stati finiti utilizzata per la gestione dell'inseguimento del marker.

4 Test e miglioramenti futuri

La fase di test è stata condotta con l'obiettivo di verificare il funzionamento dell'intero sistema, dalla ricerca iniziale del marker fino alla fase di inseguimento. Le prove sono state eseguite sul robot reale, utilizzando la mappa dell'ambiente, la localizzazione tramite Nav2 e la camera frontale per la rilevazione dell'ArUco. I test hanno permesso di valutare sia il comportamento nominale, cioè il caso in cui il marker sia visibile e il percorso sia libero, sia situazioni più critiche, come occlusioni temporanee, ostacoli lungo il percorso e perdita del marker dal campo visivo.

In particolare, sono stati considerati i seguenti aspetti:

- corretta rilevazione del marker ArUco da parte del nodo `aruco_reader`;
- corretta trasformazione della posa del marker dal frame della camera al frame globale `map`;
- corretta generazione dei goal reali, predetti e degli eventi `/target_hold` e `/target_lost` da parte di `compute_pose`;
- corretto funzionamento della FSM di inseguimento e delle sue transizioni principali;
- capacità del robot di interrompere la ricerca ed entrare nella fase di follow appena il marker viene rilevato;
- capacità del robot di aggiornare il goal durante l'inseguimento, mantenendo il movimento verso il marker;
- comportamento del sistema in caso di brevi occlusioni del marker, con utilizzo della predizione tramite filtro di Kalman;
- comportamento dello stato `LOST`, in cui il robot esegue una rotazione locale per tentare di riacquisire il marker;
- capacità di Nav2 di ricalcolare il percorso in presenza di ostacoli imprevisti lungo la traiettoria;
- corretto funzionamento della fase di esplorazione basata su waypoint, route offline e scan angles.

I test hanno evidenziato l'importanza di separare chiaramente la logica decisionale dalla navigazione. La FSM si occupa di decidere quale goal seguire e come reagire alla perdita del marker, mentre il Behavior Tree di Nav2 gestisce il planning, il controllo e le recovery durante il movimento. Questa struttura ha reso il comportamento più robusto in presenza di ostacoli, poiché il `bt_navigator` può effettuare replanning dinamico durante l'esecuzione del goal.

Un altro aspetto emerso durante le prove riguarda la gestione della rilevazione ArUco. La pubblicazione immediata di ogni singolo frame può rendere il sistema troppo sensibile al rumore, mentre un filtraggio eccessivo può ridurre la reattività. Per questo motivo è stata

adottata una soluzione intermedia: le pose reali vengono pubblicate a frequenza controllata, mentre la perdita del marker genera una sola predizione tramite Kalman. In questo modo si riduce il rischio di inviare al robot goal instabili o troppo ravvicinati.

Le prove sulla fase di esplorazione hanno permesso di confrontare diverse strategie di visita dei waypoint. La mappa estesa è stata affrontata tramite route calcolate offline con un'ottimizzazione MILP, mentre per la mappa ridotta sono state utilizzate route definite manualmente, più adatte a visitare prima le zone ritenute più rilevanti. Inoltre, gli scan angles hanno permesso di aumentare la copertura visiva della camera senza imporre rotazioni complete in ogni waypoint, riducendo il tempo totale di missione.

Nonostante il comportamento complessivo sia risultato coerente con gli obiettivi del progetto, restano alcuni possibili miglioramenti futuri. Una prima direzione riguarda il tuning del filtro di Kalman: si potrebbero introdurre soglie sul numero minimo di misure valide prima di abilitare una predizione, un tempo massimo di predizione e un limite massimo sulla distanza del goal stimato. Questo renderebbe più sicura la gestione delle occlusioni brevi, evitando di seguire stime generate da dati troppo scarsi o troppo vecchi.

Un secondo miglioramento riguarda la percezione degli ostacoli. Il sistema si basa principalmente sul lidar e sulla costmap locale; di conseguenza, ostacoli molto bassi o non rilevati correttamente possono ancora causare comportamenti non ottimali. In futuro sarebbe possibile integrare informazioni provenienti da sensori di profondità o point cloud, così da migliorare la percezione degli ostacoli non visibili dal lidar.

Infine, un'ulteriore estensione riguarda la gestione delle route durante la ricerca. Attualmente la route viene scelta all'inizio della missione e poi eseguita sequenzialmente. Una possibile evoluzione sarebbe aggiornare dinamicamente la lista dei waypoint visitati anche quando il robot, a causa di un replanning, passa vicino a punti di scansione previsti più avanti nella route. In questo modo si eviterebbero ritorni inutili e si ridurrebbe ulteriormente il tempo complessivo di esplorazione.